Detailed Test Design and Execution

Team 7

2351441 许奕

2351039 王相

2350283 康凤轩

2352746 张洛梧

2350217 陈奕玮

Detailed Test Design and Execution

1. Target Application and Selected Major Module

1.1 Target Application Context

The independent application under test in this final project is **MiniShop Checkout**, a compact e-commerce checkout prototype implemented in:

- `target_app/minishop_checkout/`

Its formal definition is recorded in:

- `final_docs/12_target_application_definition_cn.md`

`MiniShop Checkout` includes the following concrete application modules:

- cart and checkout preview orchestration
- promotion and coupon logic
- shipping fee calculation
- tax and order-total calculation
- payment-card validation
- pickup-station and recipient validation

This document focuses on one selected major module of that application, as required by the assignment.

1.2 Selected Major Module

This document treats `coupon_discount_engine` as the selected major feature/module of `MiniShop Checkout`. It belongs to the application's **Promotion service** and is not the AutoTestDesign tool itself.

Its role inside the final project is:

- to preserve a concrete and independently executable application module inside `MiniShop Checkout`
- to show that the test designs produced or refined with `ARG-Test` can be translated into black-box and white-box tests
- to provide executable evidence for a high-risk financial-rule component of the target application

This module is a strong detailed-execution target because it combines:

- valid and invalid input partitions
- multiple numeric boundaries
- interacting business rules
- explicit expected results
- a compact reference implementation suitable for white-box execution

The module is suitable for detailed execution because its expected behavior is explicit, reviewable, and rich enough to require more than one testing technique. A single nominal-path test would not be sufficient: the module has rejection rules, threshold rules, membership restrictions, and implementation branches that must be checked together.

1.3 Mapping to the Teacher-Required Detailed Chain

The assignment asks for a detailed chain from concept to evidence-based improvement. This document is explicitly organized to follow that chain:

1. **Concept**
   - define the target application and selected module
2. **Coverage item identification**
   - normalize the coupon rules into test obligations
3. **Coverage strategy and method**
   - choose EP, BVA, Decision Table, and selected white-box checks
4. **Traceability**
   - map requirement rules to concrete test cases and executable test functions
5. **Prompt design / tool use**

    - explain how `ARG-Test` was used to support design and review
6. **Result analysis**
    - interpret `pytest`, coverage, and mutation evidence
7. **Evidence-based improvement**
    - explain which obligations required deliberate reinforcement in the final suite

---

2. Requirement Basis, Scope, and Coverage Item Identification

2.1 Formalized Requirement Summary

The detailed execution document uses the following normalized rules derived from the selected requirement and the preserved reference implementation:

- R1: only one coupon may be applied to an order
- R2: unknown coupon codes must be rejected
- R3: expired coupons must be rejected
- R4: `SAVE10` requires `subtotal >= 50`
- R5: `SAVE20` requires `subtotal >= 100`
- R6: `SAVE20` also requires premium membership
- R7: `SAVE20` cannot be combined with sale items
- R8: `FREESHIP` requires `subtotal >= 30`
- R9: no-coupon input should leave subtotal and shipping unchanged

These rules define the coverage items used in the rest of the document.

2.2 Coverage Item Identification

The normalized rules were converted into explicit coverage obligations before test-case design started.

Coverage class	Coverage item	Why it matters
Valid partition	no coupon path	baseline behavior must preserve original order values
Invalid partition	multiple coupons	explicit exclusivity rule
Invalid partition	unknown coupon	unsupported input must be rejected
Invalid partition	expired coupon	temporal invalidity must be rejected
Boundary	`SAVE10` threshold at `50`	classic off-by-one risk
Boundary	`SAVE20` threshold at `100`	high-value discount threshold
Boundary	`FREESHIP` threshold at `30`	shipping fee changes discontinuously
Rule interaction	`SAVE20` + non-premium customer	premium-only condition
Rule interaction	`SAVE20` + sale items	exclusivity interaction
Expected-result obligation	correct discount math for `SAVE10` and `SAVE20`	financial correctness
Expected-result obligation	correct shipping fee effect for `FREESHIP`	output transformation correctness
White-box obligation	negative subtotal guard	implementation-level defensive branch
White-box obligation	negative shipping-fee guard	implementation-level defensive branch
White-box obligation	coupon normalization	input-normalization branch that affects accept/reject behavior

This coverage-item table is important because it bridges the raw requirement wording and the actual test suite design. It prevents the detailed document from becoming only a list of example tests without a clear coverage rationale.

2.3 Implementation Under Test

Item	Path
Target application package	`target_app/minishop_checkout/`
Promotion-service entry point	`target_app/minishop_checkout/promotion.py`
Checkout integration path	`target_app/minishop_checkout/checkout.py`
Reference implementation under detailed execution	`reference_impl/coupon_discount_engine.py`
Black-box tests	`tests/test_coupon_discount_engine_blackbox.py`
White-box tests	`tests/test_coupon_discount_engine_whitebox.py`
Seeded mutants	`reference_impl/coupon_discount_engine_mutants.py`

The detailed module is executed through the preserved `reference_impl` path so that the strongest existing `pytest`, coverage, and mutation evidence remains valid. At the same time, the application package imports the same coupon engine through the promotion-service layer, which means the detailed module is part of the chosen target application rather than an isolated helper script.

---

3. How ARG-Test Was Used in the Detailed Design

3.1 Role of the Tool in This Document

For this detailed module, `ARG-Test` is used as a **design-support and review-support tool**, not as the executable test runner. Its value in this document is to:

- structure the requirement into explicit test obligations
- propose black-box techniques suitable for the rule set
- support designer review of coverage focus
- preserve traceable artifacts from requirement to reviewed suite

The actual execution framework remains `pytest`.

3.2 Prompt-Design Intent

The generation and review process for this module is centered on the following prompt-design intent:

1. force the model to produce a structured trace rather than free text
2. make the model commit to named techniques
3. require explicit expected outputs for each test case
4. let the designer strengthen focus on high-risk obligations such as:
    - threshold boundaries
    - expired coupon handling
    - premium membership restriction
    - sale-item conflict

This prompt-design intent matters because the selected module is small enough that weak prompting would still produce fluent-looking but incomplete tests. The goal here is not to maximize verbosity, but to maximize obligation visibility.

3.3 Interactive Review Path Used Here

The final project now supports the designer-in-the-loop review cycle required by the assignment:

1. select or enter the coupon requirement
2. add technique emphasis and coverage-review notes
3. generate a structured suite
4. inspect generated cases, risk hints, and diagnostics
5. directly edit generated test cases if an obligation is weak or missing
6. export the revised suite

For this detailed module, the most important review focus was:

- threshold obligations at `30`, `50`, and `100`
- invalid financial-rule combinations

- explicit expected outputs rather than vague “should fail” wording
- implementation branches that are easy to miss from the requirement text alone

---

4. Test Environment and Tooling

4.1 Tooling

Tool	Purpose
`ARG-Test`	structured requirement analysis, black-box suite generation, and review support
`pytest`	execute black-box and white-box test cases
`coverage.py`	collect statement and branch coverage
mutant functions	demonstrate defect-detection usefulness

`pytest` was selected because the reference implementation is a compact Python module and the expected outcomes can be expressed directly with assertions. `coverage.py` was added to connect the test design with executable evidence. Branch coverage is especially useful here because coupon logic is implemented through conditionals, not only straight-line statements.

4.2 Commands Used

```
python -m pytest tests\test_coupon_discount_engine_blackbox.py
tests\test_coupon_discount_engine_whitebox.py -q
python -m pytest tests -q
python -m coverage run --branch -m pytest tests -q
python -m coverage report -m reference_impl\coupon_discount_engine.py
python -m coverage xml -o final_docs\execution_evidence\coupon_discount_engine_coverage.xml
python -m coverage xml -o final_docs\execution_evidence\coupon_discount_engine_branch_coverage.xml
```

4.3 Execution Scope

The selected module is executed at three nested scopes:

1. **module-specific test scope**
   - black-box and white-box tests for `coupon_discount_engine`
2. **repository regression scope**
   - all current repository tests
3. **application relevance scope**
   - `MiniShop Checkout` imports the same coupon logic through the promotion-service path and the checkout preview smoke tests

This separation is helpful because it shows both:

- narrow evidence for the selected module
- broader regression stability for the repository

---

5. Black-Box Test Design

The black-box design was derived from the requirement rules without relying on internal implementation details. Three techniques are used together because they cover different risks. Equivalence Partitioning separates valid and invalid classes. Boundary Value Analysis targets monetary thresholds where off-by-one or inclusive/exclusive mistakes are likely. Decision Table Testing captures business-rule combinations, especially for `SAVE20`, where multiple conditions interact.

5.1 Equivalence Partitioning

Partition ID	Partition type	Representative condition	Designed test(s)	Expected result
EP1	valid	no coupon provided	`BB01`	accept and keep subtotal/shipping unchanged
EP2	invalid	more than one coupon provided	`BB02`	reject with one-coupon-only reason
EP3	invalid	unknown coupon code	`BB03`	reject with unknown-coupon reason
EP4	invalid	expired coupon	`BB04`	reject with expired-coupon reason
EP5	invalid	premium-only coupon used by non-premium customer	`BB07`	reject with membership reason
EP6	invalid	restricted coupon combined with sale items	`BB08`	reject with sale-item restriction reason
EP7	valid	`SAVE20` with all preconditions satisfied	`BB09`	accept and apply 20% discount

5.2 Boundary Value Analysis

Boundary ID	Threshold	Below	On boundary	Above / valid representative	Designed test(s)
B1	`SAVE10` subtotal `50`	`49`	`50`	`60`	`BB05`, `BB06`, `WB05`
B2	`SAVE20` subtotal `100`	`99`	`100`	`120`	`WB03`, `BB09`
B3	`FREESHIP` subtotal `30`	`29`	`30`	`80` with no coupon	`WB04`, `BB10`, `BB01`

5.3 Decision Table

Rule	Coupon	Threshold met	Premium member	Sale items present	Expired	Expected outcome
D1	none	N/A	N/A	N/A	N/A	accept, no change
D2	`SAVE10`	no	N/A	N/A	no	reject
D3	`SAVE10`	yes	N/A	N/A	no	accept, 10% discount
D4	`SAVE20`	yes	no	no	no	reject
D5	`SAVE20`	yes	yes	yes	no	reject
D6	`SAVE20`	yes	yes	no	no	accept, 20% discount
D7	`FREESHIP`	no	N/A	N/A	no	reject
D8	`FREESHIP`	yes	N/A	N/A	no	accept, shipping becomes zero

5.4 Expected-Result Synthesis

The assignment explicitly asks for expected-result generation, so the detailed suite does not stop at selecting inputs. Each main case also has an explicit oracle rationale.

Case type	Oracle logic
`SAVE10` valid	subtotal becomes `subtotal - 10%`, shipping unchanged
`SAVE20` valid	subtotal becomes `subtotal - 20%`, shipping unchanged
`FREESHIP` valid	subtotal unchanged, shipping becomes `0.0`
invalid coupon case	status becomes rejected, subtotal and shipping remain original values
no-coupon case	status accepted, no discount applied, original values preserved

This explicit oracle layer is important because financial-rule suites are not persuasive if they only check status flags and ignore output values.

5.5 Executable Black-Box Cases

Test ID	`pytest` function	Main technique	Covered rule / purpose
BB01	`test_no_coupon_keeps_order_values`	EP	no-coupon valid partition
BB02	`test_multiple_coupons_are_rejected`	EP	one-coupon-only rejection
BB03	`test_unknown_coupon_is_rejected`	EP	unknown coupon rejection
BB04	`test_expired_coupon_is_rejected`	EP	expired coupon rejection
BB05	`test_save10_boundary_below_threshold_is_rejected`	BVA	`SAVE10` just below threshold
BB06	`test_save10_boundary_on_threshold_is_accepted`	BVA	`SAVE10` exact threshold acceptance
BB07	`test_save20_requires_premium_membership`	EP / decision table	premium requirement
BB08	`test_save20_with_sale_items_is_rejected`	EP / decision table	sale-item restriction
BB09	`test_save20_valid_case_applies_discount`	decision table	valid `SAVE20` acceptance
BB10	`test_freeship_threshold_on_boundary_sets_shipping_to_zero`	BVA	`FREESHIP` exact threshold acceptance

5.6 Requirement-to-Test Traceability

The following traceability matrix connects the requirement rules, coverage ideas, selected techniques, executable test IDs, and expected behavior. This is the main bridge between the abstract test design and the concrete `pytest` implementation.

Requirement	Coverage item	Technique	Test ID(s)	`pytest` function(s)	Expected behavior
R1	more than one coupon	EP	`BB02`	`test_multiple_coupons_are_rejected`	reject multiple coupons
R2	unknown coupon code	EP	`BB03`	`test_unknown_coupon_is_rejected`	reject unknown code
R3	expired coupon	EP	`BB04`	`test_expired_coupon_is_rejected`	reject expired coupon
R4	`SAVE10` below/on threshold	BVA	`BB05`, `BB06`, `WB05`	`test_save10_boundary_below_threshold_is_rejected`, `test_save10_boundary_on_threshold_is_accepted`, `test_coupon_normalization_accepts_mixed_case_and_spacing`	reject below 50, accept at 50 or above
R5	`SAVE20` threshold	BVA / decision table	`WB03`, `BB09`	`test_save20_below_threshold_keeps_original_values`, `test_save20_valid_case_applies_discount`	reject below 100, accept when all conditions hold
R6	premium membership required	EP / decision table	`BB07`	`test_save20_requires_premium_membership`	reject non-premium customer
R7	sale-item restriction	EP / decision table	`BB08`	`test_save20_with_sale_items_is_rejected`	reject `SAVE20`

Requirement	Coverage item	Technique	Test ID(s)	`pytest` function(s)	Expected behavior
					with sale items
R8	`FREESHIP` threshold	BVA	`WB04`, `BB10`	`test_freeship_below_threshold_is_rejected`, `test_freeship_threshold_on_boundary_sets_shipping_to_zero`	reject below 30, set shipping to zero at 30
R9	no coupon path	EP	`BB01`	`test_no_coupon_keeps_order_values`	accept and keep values unchanged

6. White-Box Test Design

6.1 White-Box Objectives

The white-box design complements the black-box suite by checking implementation branches that may not be fully justified by a single external rule. For example, the negative value guards are implementation-level safety checks, while coupon normalization verifies that the function handles user-facing input variations such as whitespace and mixed letter case.

The white-box design targets:

- input guard branches for negative values
- all coupon dispatch branches
- rejection branches for each invalid condition
- acceptance branches for `SAVE10`, `SAVE20`, and `FREESHIP`
- normalization behavior for mixed-case and whitespace coupon input

6.2 Branch-to-Test Mapping

White-box obligation	Designed test(s)
`subtotal < 0` guard raises `ValueError`	`WB01`
`shipping_fee < 0` guard raises `ValueError`	`WB02`
`SAVE20` threshold-reject branch	`WB03`
`FREESHIP` threshold-reject branch	`WB04`
coupon normalization branch	`WB05`
`SAVE10` accept path	`BB06`, `WB05`
`SAVE20` accept path	`BB09`
`FREESHIP` accept path	`BB10`

6.3 Executable White-Box Cases

Test ID	`pytest` function	Covered white-box purpose
WB01	`test_negative_subtotal_raises_value_error`	negative subtotal guard
WB02	`test_negative_shipping_fee_raises_value_error`	negative shipping fee guard
WB03	`test_save20_below_threshold_keeps_original_values`	`SAVE20` threshold rejection branch
WB04	`test_freeship_below_threshold_is_rejected`	`FREESHIP` rejection branch
WB05	`test_coupon_normalization_accepts_mixed_case_and_spacing`	normalization and `SAVE10` valid path

6.4 White-Box Adequacy Interpretation

The white-box layer matters for two reasons:

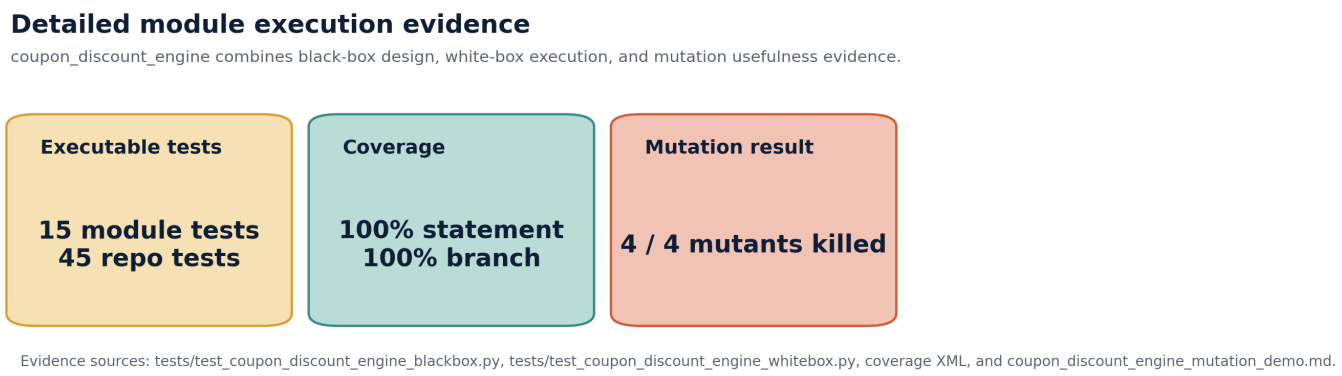
1. some implementation obligations are not visible in the requirement text, such as defensive guards for negative numbers
2. some user-facing behaviors, such as coupon normalization, are easiest to justify through implementation-path evidence

Together, the black-box and white-box tests form a layered design:

- the black-box part proves that the observable business rules are represented
- the white-box part proves that the implementation paths behind those rules are executed

---

7. Execution Results



7.1 Summary of Observed Results

Item	Observed result
Module-specific executable tests	`15 passed`
Repository regression suite at report-preparation time	`45 passed`
Statement coverage on the reference module	`100%`
Branch coverage on the reference module	`100%`
Mutation result	`4 / 4 mutants killed`

7.2 Coverage Summary

```
Name                                    Stmts   Miss Branch BrPart  Cover
reference_impl\coupon_discount_engine.py     51      0     26      0   100%
TOTAL                                        51      0     26      0   100%
```

Coverage interpretation:

- black-box design is strong enough to exercise the functional rule structure
- white-box design closes the remaining branch obligations
- no branch in the selected reference implementation remains unexecuted

7.3 Result Analysis

The detailed execution result is strong for four reasons.

First, the black-box suite is not superficial. It covers valid partitions, invalid partitions, multiple thresholds, and interacting rule conditions such as premium membership plus sale-item restrictions.

Second, the expected-result layer is explicit. The suite does not stop at “accepted” or “rejected”; it also checks financial output values such as discounted subtotal and shipping fee.

Third, the white-box suite is not decorative. It exercises the explicit negative-input guards and rejection/acceptance branches that are easy to overlook in a purely requirement-level discussion.

Fourth, the combined suite is compact. With only 15 module-focused cases, it achieves complete statement and branch coverage on the selected implementation, which is a good tradeoff between completeness and maintainability for this module-level validation.

7.4 Defect Classes Detectable by This Suite

The final suite is expected to detect at least the following defect classes:

- off-by-one threshold defects
- missing exclusivity checks
- premium-membership authorization defects
- sale-item restriction defects
- output-value calculation defects
- normalization/format handling defects
- missing defensive guards for negative numeric input

---

8. Evidence-Based Improvement Loop

The assignment explicitly asks for evidence-based improvement rather than a one-shot generated suite. This section explains how the final coupon suite goes beyond an initial nominal-path design.

8.1 Obligations That Needed Reinforcement

The highest-value reinforcement points for this module were:

- explicit rejection of multiple coupons
- explicit expired-coupon rejection
- exact threshold checks at 30, 50, and 100
- premium-member restriction separated from threshold failure
- sale-item restriction separated from membership failure
- normalization behavior for mixed-case and whitespace coupon input

These are exactly the kinds of obligations that plain generation can under-specify if the designer only accepts the first plausible-looking suite.

8.2 Concrete Improvements Preserved in the Final Suite

The final reviewed suite deliberately preserves the following cases because they materially improve coverage quality:

Improvement focus	Final case(s) preserved	Why it matters
one-coupon-only enforcement	BB02	closes explicit exclusivity rule
expired-coupon invalidity	BB04	closes time-sensitive invalid case
threshold below/on distinctions	BB05, BB06, WB03, WB04, BB10	prevents off-by-one coverage gaps
interaction of premium and sale-item rules	BB07, BB08, BB09	avoids collapsing distinct rejection causes into one vague test
normalization branch	WB05	preserves user-input variation and implementation-path evidence

8.3 Why This Counts as Real Improvement

The final suite is better than a superficial first-pass suite because:

- it distinguishes different invalid causes rather than merging them
- it makes the numeric boundaries explicit
- it checks both state/status outputs and financial outputs
- it connects requirement-visible obligations with implementation-visible branches

This is the main reason the detailed module is a credible course-project execution package rather than an illustrative toy example.

---

9. Mutation-Based Usefulness Demonstration

The evaluation also includes defect-seeded usefulness evidence. Four representative mutants were created:

- a mutant that allows multiple coupons
- a mutant that rejects `SAVE10` exactly at subtotal `50`
- a mutant that ignores the `SAVE20` sale-item restriction
- a mutant that rejects `FREESHIP` exactly at subtotal `30`

Observed outcome:

- `4 / 4` mutants were killed

Mutation-to-test mapping:

Mutant	Killed by
`multiple_coupons_allowed`	`BB02`, `BB08`
`save10_boundary_bug`	`BB06`
`save20_sale_item_bug`	`BB08`
`freeship_boundary_bug`	`BB10`

This is important because it shows that the detailed suite is not only structurally complete. It is also effective at detecting realistic logic defects that correspond to the selected business rules and boundaries.

---

10. Relevance to the Full Target Application

The selected module is not detached from the target application. Its relevance to `MiniShop Checkout` is concrete:

- `target_app/minishop_checkout/promotion.py` imports the same coupon engine
- `target_app/minishop_checkout/checkout.py` consumes coupon results inside the end-to-end preview flow
- application smoke tests already verify checkout scenarios that depend on coupon behavior

Two examples from the current smoke suite are especially relevant:

Application smoke scenario	Why it matters for the selected module
`test_checkout_preview_applies_coupon_and_computes_tax`	proves that coupon logic participates correctly in the preview pipeline, including tax and total
`test_checkout_preview_reports_invalid_payment_without_breaking_order_math`	shows that coupon calculation remains stable even when another validation area is failing

This confirms that the detailed module evidence is not isolated. It strengthens confidence in the promotion part of the real chosen target application.

---

11. Evidence Paths

Primary evidence files:

- `final_docs/execution_evidence/coupon_discount_engine_execution_summary.md`
- `final_docs/execution_evidence/coupon_discount_engine_coverage.xml`
- `final_docs/execution_evidence/coupon_discount_engine_branch_coverage.xml`
- `final_docs/execution_evidence/coupon_discount_engine_mutation_demo.md`
- `tests/test_coupon_discount_engine_blackbox.py`
- `tests/test_coupon_discount_engine_whitebox.py`
- `tests/test_minishop_checkout_smoke.py`

12. Conclusion

The `coupon_discount_engine` module is supported by a complete detailed-design and execution chain. The module is covered by multiple black-box techniques, executable white-box tests, complete statement and branch coverage, and a successful mutation demonstration. This makes it a credible detailed anchor for the overall `MiniShop Checkout` validation package rather than a merely illustrative example.

Most importantly, the evidence chain is complete:

- requirement rules are transformed into explicit coverage items
- coverage items are mapped to black-box and white-box strategies
- those strategies are traced to executable `pytest` functions
- the tests are run against the selected `MiniShop Checkout` promotion module
- the resulting suite is validated with coverage and mutation evidence
- the final suite is justified as a reviewed and improved result rather than a one-shot output

The result is a traceable module-level validation package that connects test case design, test tool implementation, and test result analysis in the exact spirit required by the assignment.